Լաբորատոր աշխատանք 10

Դինամիկ անդամներով դասեր և դինամիկ զանգվածի կառավարում

Աշխատանքի նպատակ.

Սովորել C++ ծրագրավորման լեզվում ստեղծել դասեր, որոնք օգտագործում են դինամիկ զանգվածներ՝ տվյալների պահպանման և մշակման համար: Ծանոթանալ լռելյայն և պարամետրերով կոնստրուկտորների, դեստրուկտորի և դինամիկ հիշողության կառավարման կիրառմանը: Սովորել ստեղծել դատարկ օբյեկտ, դինամիկ զանգվածում ավելացնել և հեռացնել տարրեր, ինչպես նաև անվտանգ կերպով փոփոխել զանգվածի չափը: Զարգացնել թվային տվյալների մշակման և ինտեգրալի մոտավոր հաշվարկի հմտությունները:

Խնդրի պահանջ.

Ստեղծել `IntegralStudy` դաս՝ հավասարահեռ կետերում ֆունկցիայի `y` արժեքների զանգվածը և քայլի երկարությունը՝ `h`-ը պահելու համար: Դասի `private` բաժնում սահմանել `double*` դինամիկ զանգված՝ `y` արժեքները պահպանելու համար, ինչպես նաև զանգվածի տարրերի քանակը՝ `n`, և պարամետրը՝ `p`: Քանի որ տվյալ տարբերակում `p=h`, այն օգտագործվում է որպես քայլի երկարություն:Դասում իրականացնել լռելյայն կոնստրուկտոր, որը ստեղծում է դատարկ օբյեկտ: Դատարկ օբյեկտի դեպքում `n=0`, դինամիկ զանգվածը չի պարունակում տարրեր, իսկ լռելյայն `p` արժեքը սահմանվում է `1`:Իրականացնել պարամետրերով կոնստրուկտոր, որը թույլ է տալիս ստեղծել օբյեկտ տրված `p` արժեքով և սկզբնական `y` արժեքների զանգվածով:Իրականացնել դեստրուկտոր՝ դինամիկ հիշողությունն ազատելու համար:Դասում իրականացնել `append()` մեթոդ, որը նոր արժեք է ավելացնում զանգվածի վերջում: Զանգվածի չափը փոխելիս պետք է ստեղծել նոր դինամիկ զանգված, պատճենել հին արժեքները, ավելացնել նոր արժեքը, այնուհետև ազատել հին զանգվածը:Իրականացնել `removeAt()` մեթոդ, որը տրված ինդեքսով հեռացնում է զանգվածի տարրը: Հեռացումից հետո զանգվածի չափը պետք է նվազի մեկով, իսկ մնացած արժեքները պահպանվեն ճիշտ հերթականությամբ:Տվյալների համար թույլատրվում են միայն վերջավոր իրական թվեր, իսկ `h`-ի արժեքը պետք է լինի դրական:

Տրված տվյալները.

Տրված է հավասարահեռ կետերում ֆունկցիայի արժեքների հետևյալ զանգվածը.`y = {1, 2, 4, 3}`իսկ քայլի երկարությունը՝`h = 0.5`Ինդեքսները սկսվում են `0`-ից:

Հաշվարկներ.

Պետք է մոտավորապես հաշվարկել ինտեգրալը հետևյալ բանաձևով.**I ≈ h · ((y[0] + y[n-1]) / 2 + Σy[i])**որտեղ գումարը կատարվում է `i=1`-ից մինչև `n-2` միջակայքում:

Բացի ինտեգրալի մոտավոր հաշվարկից, անհրաժեշտ է գտնել `y` զանգվածի ամենամեծ արժեքի ինդեքսը և որոշել `y` արժեքների միջին թվաբանականից մեծ արժեքների քանակը:

Իրականացում.

Լռելյայն կոնստրուկտորը պետք է ստեղծի դատարկ `IntegralStudy` օբյեկտ՝ `n=0` և `p=1` արժեքներով:Դատարկ օբյեկտում `append()` մեթոդի միջոցով պետք է հերթականությամբ ավելացնել փորձնական զանգվածի բոլոր տարրերը՝`1, 2, 4, 3`Ընդ որում, առաջին օբյեկտի լռելյայն `p=1` արժեքը պետք է պահպանվի:Առանձին պետք է ստեղծել երկրորդ օբյեկտ՝ փորձնական տվյալների `p` արժեքով, այսինքն՝`p=h=0.5`: Առաջին օբյեկտից անհրաժեշտ է հեռացնել `1` ինդեքսով տարրը: Քանի որ `y[1]=2`, հեռացումից հետո առաջին օբյեկտի զանգվածը պետք է լինի՝`{1, 4, 3}`:Այնուհետև երկու օբյեկտների համար պետք է կատարել բոլոր անհրաժեշտ հաշվարկները:

Տեսական մաս.

Սահմանում ենք `IntegralStudy` դասը՝ դինամիկ `double` զանգվածով, տարրերի քանակով `n` և պարամետրով `p`: Լռելյայն կոնստրուկտորը ստեղծում է դատարկ օբյեկտ և սահմանում `p=1`:Դատարկ օբյեկտի դեպքում դինամիկ զանգվածը չի պարունակում տարրեր, իսկ `n=0`:Պարամետրերով կոնստրուկտորը ստանում է `p` արժեքը և սկզբնական `y` արժեքների զանգվածը:Դեստրուկտորը ազատում է դինամիկ զանգվածի համար հատկացված հիշողությունը:`append()` մեթոդի միջոցով նոր արժեք ավելացնելիս ստեղծվում է նոր զանգված, հին արժեքները տեղափոխվում են նոր զանգված, ավելացվում է նոր տարրը, այնուհետև հին զանգվածը ջնջվում է: `removeAt()` մեթոդի միջոցով տրված ինդեքսով տարրը հեռացվում է, և մնացած տարրերը տեղափոխվում են նոր զանգված:Ինտեգրալի մոտավոր արժեքը հաշվարկվում է տրված բանաձևով՝ օգտագործելով առաջին և վերջին արժեքների կես գումարը և միջանկյալ արժեքների գումարը:Ստեղծվում է մեթոդ՝ `y` զանգվածի ամենամեծ արժեքի ինդեքսը գտնելու համար:Ստեղծվում է մեթոդ՝ `y` արժեքների միջին թվաբանականը հաշվելու համար:Ստեղծվում է մեթոդ՝ միջին արժեքից մեծ տարրերի քանակը որոշելու համար: Եթե զանգվածը դատարկ է, ամենամեծ արժեքի ինդեքսը սահմանվում է `-1`, իսկ միջինից մեծ արժեքների քանակը՝ `0`:Եթե ինտեգրալի հաշվարկի համար տվյալների քանակը բավարար չէ, պետք է արտածել «Տվյալները բավարար չեն» հաղորդագրությունը:Իրական թվերի համեմատության ժամանակ անհրաժեշտության դեպքում օգտագործվում է 10^{-9} ճշտություն:Զանգվածի փոփոխման մեթոդները պետք է ստուգեն մուտքային արժեքների և ինդեքսների ճիշտ լինելը:Անվավեր արժեքի կամ ինդեքսի դեպքում տվյալների նախկին վիճակը պետք է պահպանվի:Հաշվարկային մեթոդները չպետք է փոփոխեն օբյեկտի տվյալները:

Առաջադրանք.

Կազմել C++ ծրագիր, որը կկիրառի `IntegralStudy` դասը՝ հավասարահեռ կետերում տրված ֆունկցիայի `y` արժեքների հիման վրա ինտեգրալի մոտավոր հաշվարկ կատարելու համար:Ծրագիրը պետք է օգտագործի դինամիկ զանգված՝ `y` արժեքները պահպանելու նպատակով: Անհրաժեշտ է իրականացնել լռելյայն և պարամետրերով կոնստրուկտորներ, դեստրուկտոր, ինչպես նաև դինամիկ զանգվածի չափի անվտանգ փոփոխման մեթոդներ:Ծրագիրը պետք է ստեղծի դատարկ օբյեկտ՝ `p=1` լռելյայն

արժեքով, `append()` մեթոդի միջոցով ավելացնի `y={1,2,4,3}` զանգվածի տարրերը, այնուհետև առաջին օբյեկտից հեռացնի `index 1`-ի տարրը:Առանձին պետք է ստեղծել երկրորդ օբյեկտ՝ `p=h=0.5` արժեքով և սկզբնական `y={1,2,4,3}` զանգվածով:Երկու օբյեկտների համար անհրաժեշտ է հաշվել ինտեգրալի մոտավոր արժեքը, գտնել `y`-ի ամենամեծ արժեքի ինդեքսը և որոշել միջին թվաբանականից մեծ արժեքների քանակը:Ծրագիրը պետք է ապահովի դինամիկ հիշողության ճիշտ կառավարում, մուտքային տվյալների ստուգում և զանգվածի փոփոխման ժամանակ հին տվյալների պահպանում:

```
#include <iostream>
#include <cmath>
using namespace std;
class IntegralStudy {
private:
   double* data;
   int n;
   double p;
public:
   // Լռելյայն constructor
   IntegralStudy() {
      data = nullptr;
      n = 0;
      p = 1;
   }
   // Constructor տվյալներով
   IntegralStudy(double p, int n, double values[]) {
      data = nullptr;
      this->n = 0;
      this->p = 1;
      if (!isfinite(p) || p <= 0) {
         return;
      }
      if (n < 3 || n > 30) {
         return;
      }
      for (int i = 0; i < n; i++) {
         if (!isfinite(values[i])) {
            return;
         }
      }
      this->p = p;
      this->n = n;
      data = new double[n];
      for (int i = 0; i < n; i++) {
         data[i] = values[i];
      }
   }
```

```
// Destructor
~IntegralStudy() {
   delete[] data;
}
// Նոր տարր ավելացնել
bool append(double value) {
   if (!isfinite(value)) {
      return false;
   }
   if (n >= 30) {
      return false;
   }
   double* newData = new double[n + 1];
   for (int i = 0; i < n; i++) {
      newData[i] = data[i];
   }
   newData[n] = value;
   delete[] data;
   data = newData;
   n++;
   return true;
}

// Տրված ինդեքսով տարրը հեռացնել
bool removeAt(int index) {
   if (index < 0 || index >= n) {
      return false;
   }
   double* newData = nullptr;
   if (n - 1 > 0) {
      newData = new double[n - 1];
      int j = 0;
      for (int i = 0; i < n; i++) {
         if (i != index) {
            newData[j] = data[i];
            j++;
         }
      }
   }
   delete[] data;
   data = newData;
   n--;
   return true;
}

// Տպել տվյալները
```

```
void print() const {
    cout << "n = " << n << endl;
    cout << "p = " << p << endl;
    if (n == 0) {
        cout << "Զանգվածը դատարկ է" << endl;
        return;
    }
    cout << "y = {";
    for (int i = 0; i < n; i++) {
        cout << data[i];
        if (i < n - 1) {
            cout << ", ";
        }
    }
    cout << "}" << endl;
}
// Integral-ի հաշվարկ
double calculateIntegral() const {
    double sum = (data[0] + data[n - 1]) / 2.0;
    for (int i = 1; i < n - 1; i++) {
        sum += data[i];
    }
    return p * sum;
}
// Ամենամեծ y-ի ինդեքսը
int findMaxIndex() const {
    if (n == 0) {
        return -1;
    }
    int maxIndex = 0;
    for (int i = 1; i < n; i++) {
        if (data[i] > data[maxIndex] + 1e-9) {
            maxIndex = i;
        }
    }
    return maxIndex;
}
// y-երի միջինը
double calculateAverage() const {
    if (n == 0) {
        return 0;
    }
    double sum = 0;
    for (int i = 0; i < n; i++) {
        sum += data[i];
    }
```

```
        return sum / n;
    }
    // Միջինից մեծ արժեքների քանակը
    int countAboveAverage() const {
        if (n == 0) {
            return 0;
        }
        double average = calculateAverage();
        int count = 0;
        for (int i = 0; i < n; i++) {
            if (data[i] > average) {
                count++;
            }
        }
        return count;
    }
    // Բոլոր հաշվարկները
    void showCalculations() const {
        cout << "------------------------" << endl;
        if (n < 2) {
            cout << "Integral: Տվյալները բավարար չեն" << endl;
        } else {
            cout << "Integral = " << calculateIntegral() << endl;
        }
        cout << "Ամենամեծ y-ի ինդեքս = "
             << findMaxIndex() << endl;
        cout << "Միջինից մեծ արժեքների քանակ = "
             << countAboveAverage() << endl;
    }
};

int main() {
    double values[] = {1, 2, 4, 3};
    double h = 0.5;
    int n = 4;
    // ----------------------------------
    // Առաջին օբյեկտ
    // Լռելյայն p = 1
    // ----------------------------------
    IntegralStudy first;
    cout << "Առաջին օբյեկտը սկզբում:" << endl;
    first.print();
    // append-ով ավելացնում ենք բոլոր տարրերը
    for (int i = 0; i < n; i++) {
        first.append(values[i]);
    }
```

```
    cout << endl;
    cout << "Առաջին օբյեկտը append-ից հետո:" << endl;
    first.print();
    first.showCalculations();
    // index 1-ի տարրի հեռացում
    first.removeAt(1);
    cout << endl;
    cout << "Առաջին օբյեկտը index 1-ի տարրը հեռացնելուց հետո:" << endl;
    first.print();
    first.showCalculations();
    // ----------------------------------
    // Երկրորդ օբյեկտ
    // p = h = 0.5
    // ----------------------------------
    IntegralStudy second(h, n, values);
    cout << endl;
    cout << "Երկրորդ օբյեկտը:" << endl;
    second.print();
    second.showCalculations();
    return 0;
}
```